

```
/*
# InterNexus Copilot Assistant Preprompt

## Core Features
- 3D Virtual HQ Tours: Implement a Three.js/WebGL 3D environment for interactive tours of virtual offices, labs, and meeting spaces. Three.js is a robust JS 3D library (built on WebGL) and has been used for Matterport-style walkthroughs 1 2. Consider React Three Fiber (Three.js in React) or Babylon.js as alternatives for scene rendering 3.
- Project & Resource Dashboards: Build data-driven dashboards (using React or Vue) to track projects, resources, and KPIs. Utilize React components (charts, tables, forms) from libraries like Recharts/D3 or UI kits (e.g. Ant Design) for visualization and interactivity 4 5. Manage state with React Context or Redux for scalability 6. Design dashboards per UX best practices (logical grouping, drill-downs, clear key metrics) 7 8.
- Real-time Collaboration: Enable live collaboration (chat, shared whiteboards, coding) via WebSockets or Firebase. Use Socket.IO (Node.js) or native WebSockets for low-latency bidirectional updates 9. Alternatively, Firebase Realtime Database automatically syncs JSON data across all clients in milliseconds 10. Architect with a pub/sub model (rooms or channels) to broadcast events (e.g., document edits).
- Impact Metrics Visualization: Create interactive charts and metrics to show research impact (e.g. publications, funding, engagement). Emphasize top-level KPIs (highlight with size/color) and allow filtering or drill-down for details 8. Ensure dashboards are accessible (avoid color-only cues, use ARIA labels, support keyboard navigation) 11. Use responsive design (CSS grid, Bootstrap/Tailwind) so info displays well on all devices 12 13.
- Global Partner Maps & Directory: Integrate a world map showing partner institutions. Use React-based map libraries: e.g., React Simple Maps (D3-geo/topojson) or React Leaflet (Leaflet OpenStreetMap) for interactive SVG maps 14 15. For rich features, google-map-react (Google Maps API) provides satellite/3D tiles and overlays 16. Include zoomable markers and searchable partner directory, ensuring smooth panning and clustering for large data sets.
- AI-Powered Recommendations: Embed AI for decision support. For example, use TensorFlow.js to run ML models in-browser (TensorFlow.js lets you build/train models in JS) 17. You might train a recommendation model to suggest tasks or collaborators. Alternatively, call OpenAI's GPT via API to auto-generate project suggestions or summaries. Use Python (with Django) plus TensorFlow or PyTorch on the backend if needed (Django is popular for AI-driven apps) 18 19.
- Admin Tools & Notifications: Provide an admin interface for user onboarding, roles, and system alerts. Leverage built-in auth (e.g. Firebase Auth or Auth0) and create React/Vue components for user management forms. Implement a notification system (email or in-app) using backend services. Follow solid UI
```

patterns: reuse form components, validate input, and design a separate admin dashboard.

Tech Stack

- **Frontend:** Three.js (WebGL) for 3D scenes + React or Vue.js for UI components. Use Figma/Adobe XD designs to guide CSS styling and component layouts. Componentize the UI (each screen/feature as a reusable component) ²⁰.
- **Backend:** Node.js (Express, Socket.IO) *or* Django (Python). Node.js excels for non-blocking real-time services ²¹; Django offers rapid development and a strong admin/ORM ¹⁸ ¹⁹. Use REST or GraphQL APIs to communicate with the frontend.
- **Realtime Sync:** Firebase Realtime Database (or Firestore) can auto-sync data to clients ¹⁰, or use a WebSocket server (Socket.IO) for pub/sub messaging. Use serverless functions or a Node service to relay events if not using Firebase.
- **AI Integration:** TensorFlow.js (client-side ML) ¹⁷ or call OpenAI GPT (server-side) for language tasks. For example, host ML models on AWS/GCP or use TensorFlow.js in-browser for fast inference.
- **Cloud Hosting:** AWS or Google Cloud for deployment. Both offer similar VM/container services (AWS EC2 vs GCP Compute Engine) ²², managed databases, and serverless (Lambda/Cloud Run). They scale globally, so choose based on team expertise or pricing.
- **Optional VR/AR:** If adding immersive VR/AR, use Unity (export to WebGL/WebXR) or WebXR frameworks. Unity's WebXR exporter can generate browser VR content ²³. The WebXR Device API (and libraries like A-Frame or Babylon.js) lets the browser interface with VR/AR hardware ²⁴ ³.

Development Guidelines

- **Clarity & Modularity:** Write clear, well-commented code. Break logic into small, reusable functions or classes (follow DRY and SOLID principles) ²⁵ ²⁶. For example, each component or service should have a single responsibility ²⁷. Avoid "spaghetti" code; split large features into modules/files.
- **Component Reusability:** Emphasize a component-based architecture. In React/Vue, build UI as composable components with well-defined props and events. Reuse components for widgets like charts, forms, modals (the reusability accelerates development and consistency ²⁸ ²⁰).
- **Scalability:** Design state management for growth. Use React Context or Redux for shared state; local component state for isolated data ⁶. As user/data volume grows, ensure the architecture (e.g. microservices in Node or modular apps in Django) can scale. Use cloud features (auto-scaling, load balancers) on AWS/GCP.
- **UX Best Practices:** Structure the UI for clarity. Group related information, highlight key metrics (using size/color), and provide interactive filters or drill-down options ⁸ ¹³. Ensure accessibility (ARIA labels, keyboard support) and responsive design (media queries or CSS frameworks) ¹¹ ¹².
- **Code Comments:** Encourage Copilot to generate meaningful comments explaining *_why_*, not just *_what_*. Good comments clarify intent or link to docs (e.g. algorithms, APIs) ²⁹. Mark TODOs and note assumptions. Avoid redundant

```
comments that restate obvious code 29 .  
- **Suggest Alternatives:** When proposing solutions, list alternatives. For  
example, if using Three.js mention Babylon.js or A-Frame as options; React vs  
Vue; Firebase vs WebSocket; Node vs Django. Explain trade-offs briefly. This  
ensures flexible, well-justified design choices.  
- **Documentation & Testing:** Copilot should add docstrings/JSdoc for new  
functions/components and propose unit tests. Aim for high code readability and  
maintainability so future developers understand and can extend the system easily  
26 29 .  
*/
```

1 Tech Friend: tips on how to build an app with Firebase, React and Three.js | by Francisco Sierra | Medium

<https://francisco-sierra.medium.com/tech-friend-tips-on-how-to-built-an-app-with-firebase-react-and-three-js-7986717db1d4>

2 Interior virtual tour demo with bananas - Showcase - three.js forum

<https://discourse.threejs.org/t/interior-virtual-tour-demo-with-bananas/80292>

3 24 Immersive Web Developer Home

<https://immersiveweb.dev/>

4 5 6 12 20 React Dashboard Libraries Showdown: A Comprehensive Guide

<https://www.dhiwise.com/post/creating-data-visualizations-with-react-dashboard-libraries>

7 8 11 13 Effective Dashboard Design Principles for 2025 | UXPin

<https://www.uxpin.com/studio/blog/dashboard-design-principles/>

9 Realtime Collaboration App with WebSockets & Node.js - DEV Community

https://dev.to/syed_mudasseranayat_e251/realtime-collaboration-app-with-websockets-nodejs-n90

10 Firebase Realtime Database

<https://firebase.google.com/docs/database>

14 15 16 Free React Map Libraries for Google Maps, Leaflet, and SVG Maps

<https://medevel.com/react-map-library-1816/>

16 Retool Blog | The best React map libraries in 2024

<https://retool.com/blog/react-map-library>

17 Machine Learning for JavaScript Developers - TensorFlow.js

<https://www.tensorflow.org/js>

18 19 21 Django vs. Node.js: The Battle of Backend Frameworks in 2024 | by ProspexAI | Medium

<https://tarekeesa7.medium.com/django-vs-node-js-the-battle-of-backend-frameworks-in-2024-34db95ed0065>

22 Differences between AWS to Google Cloud | Google Cloud Blog

<https://cloud.google.com/blog/products/application-modernization/differences-between-aws-to-google-cloud>

23 GitHub - De-Panther/unity-webxr-export: Develop and export WebXR experiences using Unity WebGL

<https://github.com/De-Panther/unity-webxr-export>

25 27 28 Maximizing Code Reusability: Best Practices and Strategies | Graph AI

<https://www.graphapp.ai/blog/maximizing-code-reusability-best-practices-and-strategies>

26 Modular code — Quality Assurance of Code for Analysis and Research

https://best-practice-and-impact.github.io/qa-of-code-guidance/modular_code.html

29 Best practices for writing code comments - Stack Overflow

<https://stackoverflow.blog/2021/12/23/best-practices-for-writing-code-comments/>